Problem 1.1.23

ee25btech11023-Venkata Sai

March 26, 2026

1. Problem
2. Solution
   - Formula
   - Calculation
3. Plot
4. Python Code

Problem

A unity feedback control system has an open-loop transfer function

$$G(s) = \frac{K}{s(s^2 + 7s + 12)}. \tag{2.1}$$

The gain K for which $s = -1 + j1$ will lie on the root locus of this system is: (EC 2007)

1. 4
2. 5.5
3. 6.5
4. 10

Formula

We define the open-loop transfer function as

$$G(s) = K \cdot P(s) \tag{1.1}$$

where P(s) is called Plant and K is called Gain

$$P(s) = \frac{1}{s(s^2 + 7s + 12)} = \frac{1}{s(s+3)(s+4)} \tag{1.2}$$

In control theory, a point s lies on the root locus if it satisfies the characteristic equation:

$$1 + G(s)H(s) = 0 \implies 1 + KP(s)H(s) = 0 \tag{1.3}$$

$$KP(s)H(s) = -1 \tag{1.4}$$

$$|KP(s)H(s)| = 1 \tag{1.5}$$

For an unity feedback system , $H(s)$ equals 1. Then,

$$|KP(s)| = 1 \tag{1.6}$$

Calculation

In Root Locus analysis, the gain K is assumed to be a positive real number (K>0). So,

$$K|P(s)| = 1 \implies K = \frac{1}{|P(s)|} \tag{1.7}$$

From (1.2)

$$K = |s(s+3)(s+4)| \tag{1.8}$$

On substituting $s = -1 + j1$,

$$K = |(-1+j1)(-1+j1+3)(-1+j1+4)| \tag{1.9}$$

$$K = |(-1+j1)(2+j1)(3+j1)| \tag{1.10}$$

$$K = \left(\sqrt{1^2+1^2}\right)\left(\sqrt{2^2+1^2}\right)\left(\sqrt{3^2+1^2}\right) \tag{1.11}$$

$$K = \left(\sqrt{2}\right)\left(\sqrt{5}\right)\left(\sqrt{10}\right) = \sqrt{100} = 10 \tag{1.12}$$

Hence the value of K is 10.

Calculation

$$G(s) = \frac{K}{s(s^2 + 7s + 12)} = \frac{K}{s(s+3)(s+4)} \tag{1.13}$$

To find open-loop poles, we make the denominator $= 0$

$$s(s+3)(s+4) = 0 \tag{1.14}$$

Hence, the open-loop poles are 0,-3,-4
After applying the feedback, the closed loop poles are the roots of

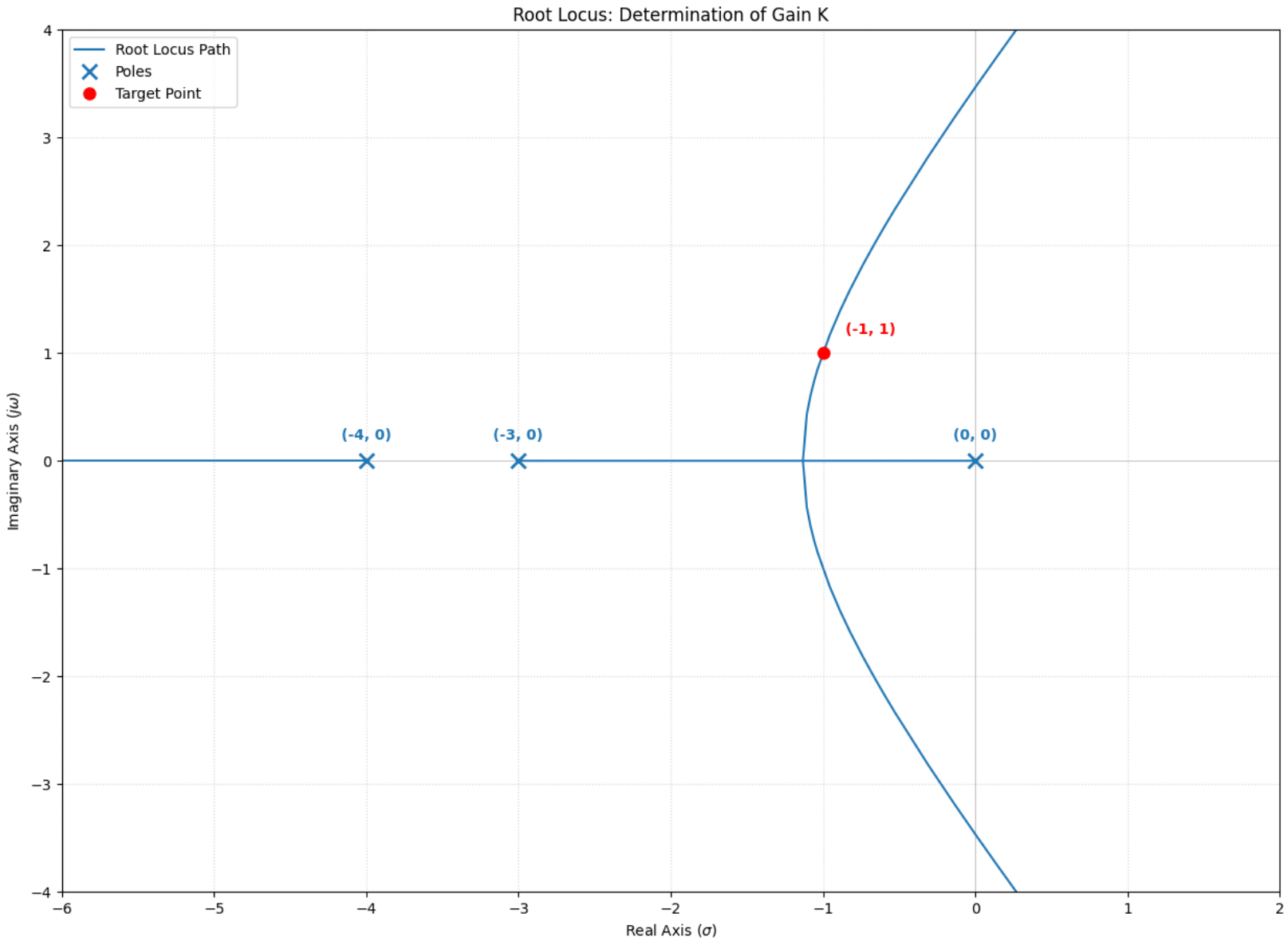
$$1 + G(s)H(s) = 0 \tag{1.15}$$

$$1 + \frac{K}{s(s+3)(s+4)} = 0 \tag{1.16}$$

$$s^3 + 7s^2 + 12s + K = 0 \tag{1.17}$$

When K=10, the roots are $-1 \pm j1$ and -5 . These are called closed loop poles.

Plot


Root Locus: Determination of Gain K
Root Locus Path
Poles
Target Point
(-1, 1)
(-4, 0)
(-3, 0)
(0, 0)
Imaginary Axis ($j\omega$)
Real Axis (σ)
4
3
2
1
0
−1
−2
−3
−4
−6
−5
−4
−3
−2
−1
0
1
2

Python Code for Plotting

```python
import numpy as np
import control as ct
import matplotlib.pyplot as plt

# G(s) = K / [s(s+3)(s+4)]
num = [1]
den = [1, 7, 12, 0] # s^3 + 7s^2 + 12s
sys = ct.TransferFunction(num, den)

# Find poles
s_target = -1 + 1j
poles = ct.poles(sys)

# K = Product of distances from each pole to the target point
distances = [np.abs(s_target - p) for p in poles]
K_calc = np.prod(distances)

# Visualization
plt.figure(figsize=(9, 6))
```

Python Code for Plotting

```
# Root Locus
roots, gains = ct.root_locus(sys, plot=False)

# Plot Root Locus Path
for i in range(roots.shape[1]):
    lbl = 'Root Locus Path' if i == 0 else
    plt.plot(np.real(roots[:, i]), np.imag(roots[:, i]), color='
        C0', lw=1.5, label=lbl, zorder=2)

# Plot Open-Loop Poles
plt.plot(np.real(poles), np.imag(poles), 'x', color='C0',
    markersize=10, mew=2, label='Poles', zorder=3)

# Plot Target Point
plt.plot(s_target.real, s_target.imag, 'ro', markersize=8, label=
    f'Target Point', zorder=4)
```

Python Code for Plotting

```
for p in poles:
    x_coord = np.real(p)
    y_coord = np.imag(p)
    # Using :g formats it cleanly without trailing zeros
    plt.text(x_coord, y_coord + 0.2, f'({x_coord:g}, {y_coord:g})
        ',
            color='C0', fontsize=10, fontweight='bold', ha='
                center')

t_x = s_target.real
t_y = s_target.imag
plt.text(t_x + 0.15, t_y + 0.15, f'({t_x:g}, {t_y:g})',
        color='red', fontsize=10, fontweight='bold', va='bottom',
            ha='left')

plt.title(Root Locus: Determination of Gain K)
plt.xlabel(Real Axis ($\sigma$))
```

Python Code for Plotting

```
plt.ylabel(Imaginary Axis ($j\omega$))
plt.axhline(0, color='black', lw=0.5, alpha=0.3, zorder=1)
plt.axvline(0, color='black', lw=0.5, alpha=0.3, zorder=1)
plt.grid(True, linestyle=':', alpha=0.5)
plt.xlim([-6, 2])
plt.ylim([-4, 4])
plt.legend(loc='upper left')
plt.tight_layout()
plt.show()

print(fTarget Point Coordinates: ({s_target.real:g}, {s_target.
    imag:g}))
print(fPole Coordinates: {[(np.real(p), np.imag(p)) for p in
    poles]})
print(fCalculated Gain K: {K_calc:.2f})
```